ABSTRACT

The objective of this project is to design and implement a **benchmarking framework for operating systems**, with a focus on analyzing and comparing different combinations of resource management policies. The framework evaluates core components of an operating system, including CPU scheduling, memory management, and disk scheduling, to determine the most effective combination for a given workload.

The system supports multiple workload types such as **CPU-bound, I/O-bound, memory-bound, and mixed workloads**. Based on user-defined parameters, the framework generates randomized data to simulate realistic execution conditions. It then applies various algorithms such as FCFS, SJF, Priority, Round Robin, FIFO, LRU, Optimal, SSTF, and SCAN across all possible combinations. Each combination is evaluated using performance metrics including **average waiting time, page faults, and seek time**.

To ensure fair comparison, the computed metrics are normalized into a unified scoring system, allowing all results to be represented within a standard range. The framework identifies the **best-performing policy combinations** and provides insights explaining their effectiveness for the selected workload. Additionally, the results are exported and visualized using R, enabling clear comparison of the top-performing strategies.
This project serves as a practical application of operating system concepts, demonstrating how different policies interact under varying workload conditions and highlighting the importance of **workload-aware decision making**.

TABLE OF CONTENTS

CHAPTER PAGE NO
ABSTRACT ---------- 1
LIST OF FIGURES ---------- 13
CHAPTERS
CHAPTER 1 – Introduction ---------- 3
CHAPTER 2 – Method ---------- 4
CHAPTER 3 – Programs ---------- 6
CHAPTER 4 – Results ---------- 10
CHAPTER 7 – Conclusion ---------- 13
REFERENCES ---------- 14

CHAPTER-1

INTRODUCTION

Operating systems are responsible for managing system resources and ensuring efficient process execution. Key components such as CPU scheduling, memory management, and disk scheduling use various algorithms like FCFS, SJF, Priority, Round Robin, FIFO, LRU, Optimal, SSTF, and SCAN. The performance of these algorithms varies depending on the workload characteristics. In most cases, these algorithms are studied individually. However, real-world operating systems rely on a combination of these policies working together. Therefore, overall system performance depends not only on individual algorithms but also on how they interact under different workloads.

A major challenge in operating system design is identifying the most suitable combination of policies for a given workload. Workloads such as CPU-bound, I/O-bound, and memory-bound processes have different requirements, and a policy that performs well in one scenario may not perform effectively in another.This project introduces a benchmarking framework for operating systems that evaluates combinations of CPU scheduling, page replacement, and disk scheduling algorithms. The system allows users to simulate different workload types by providing configurable parameters such as number of processes, burst time, memory references, and disk requests. Based on these inputs, randomized data is generated to simulate realistic conditions.

Each combination is evaluated using key metrics such as average waiting time, page faults, and seek time, which are normalized into a unified scoring system. The framework then identifies the best-performing policy combinations and provides insights into their effectiveness.This approach enables integrated evaluation of operating system policies and highlights the importance of workload-aware decision making in improving system performance.

CHAPTER-2

METHODOLOGY

The proposed system follows a structured approach to evaluate and compare different combinations of operating system policies. The methodology consists of the following stages:

1. Workload Selection and Input Configuration

The framework begins by allowing the user to select a workload type, such as CPU-bound, I/O-bound, memory-bound, or mixed. Based on the selected workload, the user provides system parameters including number of processes, burst time range, page reference length, number of disk requests, and memory frames. These inputs define the characteristics of the simulated environment.

2. Workload Generation

Using the provided parameters, the system generates randomized data to simulate realistic operating conditions. This includes generating process burst times, arrival times, priorities, memory reference strings, and disk request queues. The randomness ensures variability while maintaining the nature of the selected workload.

3. Algorithm Combination Execution

The framework applies multiple algorithms across three key components:

CPU Scheduling: FCFS, SJF, Priority, Round Robin

Page Replacement: FIFO, LRU, Optimal

Disk Scheduling: FCFS, SSTF, SCAN

All possible combinations of these algorithms are executed to evaluate their collective performance under the same workload conditions.

4. Performance Evaluation

For each combination, the system computes key performance metrics:

a. Average Waiting Time (CPU performance)
b. Page Faults (memory performance)
c. Seek Time (disk performance)

These metrics provide a comprehensive measure of system efficiency.

5. Score Normalization

Since the metrics have different scales, they are normalized to ensure fair comparison. First, individual metrics are scaled relative to their maximum values. Then, a unified score is computed and further normalized using a relative approach, where the best-performing combination is assigned a score of 1, and others are scaled accordingly.

6. Decision and Ranking

The normalized scores are used to rank all combinations. The framework identifies the top-performing policy combinations based on the highest scores. In cases where multiple combinations have similar scores, they are treated as equally optimal.

7. Insight Generation

The system generates explanations for the selected best policies based on workload characteristics and observed performance trends. This helps users understand why certain combinations perform better under specific conditions.

8. Result Export and Visualization

The final results, including all evaluated combinations and their scores, are exported to a CSV file. These results are then visualized using R, where the top-performing combinations are displayed through a horizontal bar chart for easy comparison.

CHAPTER-3

PROGRAMS

1. Main.py:

```
from config.settings import
DEFAULT_CONFIG

from workloads.cpu_bound import
generate_cpu_bound
from workloads.io_bound import
generate_io_bound
from workloads.memory_bound import
generate_memory_bound
from workloads.mixed import
generate_mixed

from engine.runner import
run_benchmark
from engine.normalizer import
normalize_results
from analysis.decision import
select_best
from analysis.insights import
generate_insights

from utils.helpers import
export_to_csv

def
get_user_config(workload_type):
    print("\nChoose Mode:")
    print("1. Default (quick run)")
    print("2. Custom input")

    mode = int(input("Enter choice:
"))

    if mode == 1:
        return DEFAULT_CONFIG

    print("\n Recommended
Parameter Guidelines:")

    if workload_type == "CPU-
bound":
        print("- High burst time")
        print("- Low disk requests")
```

```
        print("- Moderate page references")

    elif workload_type == "IO-
bound":
        print("- Low burst time")
        print("- High disk requests")
        print("- Moderate memory
usage")

    elif workload_type == "Memory-
bound":
        print("- High page reference
length")
        print("- Limited frames (to
induce page faults)")
        print("- Moderate CPU usage")

    elif workload_type == "Mixed":
        print("- Balanced values
across CPU, memory, and disk")

    print("\nEnter your custom
values:\n")

    config = {}
    config["num_processes"] =
int(input("Number of processes: "))
    config["burst_min"] =
int(input("Min burst time: "))
    config["burst_max"] =
int(input("Max burst time: "))
    config["pages_length"] =
int(input("Page reference length: "))
    config["disk_requests"] =
int(input("Disk requests: "))
    config["frames"] =
int(input("Number of frames: "))
    config["seed"] = None

    return config


def choose_workload():
    print("\nChoose Workload:")
    print("1. CPU-bound")
    print("2. IO-bound")
    print("3. Memory-bound")
    print("4. Mixed")

    choice = int(input("Enter choice
(1-4): "))

    if choice == 1:
        return generate_cpu_bound,
```

```
"CPU-bound"
    elif choice == 2:
        return generate_io_bound,
"IO-bound"
    elif choice == 3:
        return
generate_memory_bound,
"Memory-bound"
    elif choice == 4:
        return generate_mixed,
"Mixed"
    else:
        print("Invalid choice")
        exit()


def print_generated_data(data):
    print("\n Generated Workload Data")

    print("\nCPU:")
    print("Burst Time:",
data["cpu"]["burst_time"])
    print("Arrival Time:",
data["cpu"]["arrival_time"])
    print("Priority:",
data["cpu"]["priority"])

    print("\nMemory:")
    print("Pages:",
data["memory"]["pages"])
    print("Frames:",
data["memory"]["frames"])

    print("\nDisk:")
    print("Queue:",
data["disk"]["queue"])
    print("Head:",
data["disk"]["head"])


def main():
    # Step 1: Workload
    generator, workload_type =
choose_workload()

    # Step 2: Config
    config =
get_user_config(workload_type)

    # Generate workload
    data = generator(**config)

    # Print generated values
    print_generated_data(data)
```

```
    # Run benchmark
    results = run_benchmark(data)
    results = normalize_results(results)

    # Decision + insights
    best = select_best(results)
    insights =
generate_insights(data["type"], best)

    print("\nBest Policies:")
    for b in best:
        print(b)

    print("\nInsights:")
    for item in insights:
        print(f"\nPolicy:
{item['policy']}")
        for line in item["explanation"]:
            print("-", line)

    # Export CSV for R
    export_to_csv(results)

    print("\n Results exported to
results.csv")
    print(" Open plots.R in RStudio
to visualize the results")


if __name__ == "__main__":
    main()
```

CHAPTER-4

RESULTS

1. Input Parameters:

```
Choose Workload:
1. CPU-bound
2. IO-bound
3. Memory-bound
4. Mixed
Enter choice (1-4): 1

Choose Mode:
1. Default (quick run)
2. Custom input
Enter choice: 2
```

```
🔹 Recommended Parameter Guidelines:
- High burst time
- Low disk requests
- Moderate page references

Enter your custom values:

Number of processes: 1000
Min burst time: 1000
Max burst time: 2000
Page reference length: 20
Disk requests: 50
Number of frames: 5

🔹 Generated Workload Data
```

2. Randomly Generated Data based on the Input Parameters:

```
CPU:
Burst Time: [1822, 1517, 1396, 1381, 1154, 1544, 1526, 1280, 1266, 1244, 1750, 1057, 1305, 1562, 1915, 1717, 1995, 1114, 1
214, 1751, 1672, 1328, 1780, 1743, 1360, 1898, 1543, 1454, 1279, 1956, 1665, 1084, 1306, 1263, 1923, 1420, 1724, 1207, 123
2, 1936, 1315, 1463, 1062, 1064, 1785, 1562, 1158, 1311, 1538, 1572, 1501, 1002, 1863, 1851, 1844, 1681, 1210, 1614, 1117,
 1869, 1845, 1802, 1879, 1817, 1658, 1776, 1011, 1666, 1863, 1705, 1362, 1456, 1609, 1993, 1548, 1403, 1106, 1299, 1160, 1
677, 1566, 1359, 1143, 1417, 1713, 1380, 1184, 1200, 1921, 1006, 1026, 1507, 1118, 1962, 1484, 1248, 1878, 1213, 1736, 120
7, 1767, 1057, 1417, 1823, 1807, 1973, 1475, 1190, 1593, 1919, 1829, 1572, 1555, 1759, 1026, 1312, 1773, 1382, 1300, 1042,
 1129, 1895, 1014, 1849, 1977, 1472, 1357, 1334, 1306, 1516, 1329, 1779, 1682, 1235, 1566, 1145, 1257, 1495, 1147, 1569, 1
791, 1102, 1203, 1622, 1347, 1316, 1495, 1523, 1003, 1184, 1300, 1294, 1299, 1648, 1444, 1059, 1385, 1056, 1676, 1949, 151
1, 1826, 1129, 1075, 1796, 1027, 1647, 1155, 1369, 1695, 1487, 1010, 1354, 1645, 1463, 1211, 1257, 1233, 1968, 1081, 1284,
 1104, 1430, 1919, 1592, 1970, 1266, 1290, 1663, 1594, 1496, 1750, 1491, 1883, 1330, 1599, 1920, 1626, 1679, 1664, 1657, 1
702, 1134, 1330, 1878, 1795, 1195, 1451, 1633, 1984, 1500, 1900, 1773, 1498, 1953, 1886, 1544, 1302, 1374, 1134, 1465, 157
6, 1657, 1841, 1227, 1599, 1059, 1766, 1029, 1380, 1880, 1723, 1088, 1952, 1419, 1171, 1188, 1634, 1975, 1564, 1047, 1353,
 1776, 1446, 1333, 1897, 1717, 1365, 1247, 1617, 1440, 1975, 1630, 1005, 1694, 1011, 1636, 1810, 1964, 1097, 1401, 1436, 1
225, 1220, 1699, 1550, 1230, 1627, 1679, 1713, 1473, 1941, 1688, 1031, 1647, 1039, 1967, 1574, 1781, 1814, 1761, 1340, 100
0, 1304, 1740, 1906, 1276, 1227, 1107, 1305, 1161, 1332, 1894, 1966, 1574, 1763, 1436, 1689, 1476, 1158, 1708, 1367, 1556,
 1383, 1330, 1493, 1399, 1497, 1399, 1920, 1921, 1777, 1472, 1355, 1764, 1788, 1182, 1991, 1056, 1471, 1010, 1020, 1693, 1
454, 1730, 1284, 1728, 1611, 1200, 1497, 1303, 1123, 1114, 1608, 1241, 1653, 1960, 1400, 1000, 1113, 1637, 1724, 1162, 193
6, 1645, 1408, 1061, 1779, 1517, 1014, 1679, 1547, 1702, 1634, 1455, 1863, 1701, 1855, 1291, 1164, 1258, 1254, 1195, 1481,
 1411, 1178, 1453, 1664, 1716, 1134, 1174, 1763, 1160, 1882, 1196, 1389, 1309, 1148, 1078, 1617, 1101, 1907, 1344, 1132, 1
526, 1268, 1714, 1829, 1225, 1585, 1017, 1774, 1896, 1504, 1843, 1940, 1390, 1415, 1378, 1821, 1242, 1575, 1325, 1530, 163
1, 1627, 1229, 1711, 1189, 1009, 1866, 1489, 1856, 1923, 1093, 1363, 1507, 1669, 1149, 1677, 1706, 1025, 1085, 1595, 1691,
 1665, 1482, 1749, 1545, 1074, 1512, 1432, 1922, 1713, 1198, 1444, 1034, 1053, 1600, 1028, 1349, 1363, 1289, 1641, 1440, 1
688, 1567, 1998, 1773, 1415, 1107, 1411, 1905, 1646, 1431, 1203, 1735, 1403, 1947, 1215, 1120, 1859, 1797, 1233, 1085, 104
9, 1164, 1914, 1097, 1257, 1820, 1256, 1364, 1116, 1589, 1234, 1954, 1266, 1739, 1255, 1331, 1650, 2000, 1872, 1702, 1472,
 1048, 1489, 1198, 1588, 1151, 1427, 1404, 1050, 1698, 1775, 1619, 1652, 1679, 1555, 1687, 1728, 1408, 1282, 1690, 1855, 1
652, 1138, 1879, 1177, 1952, 1434, 1538, 1601, 1318, 1740, 1113, 1442, 1441, 1472, 1973, 1889, 1188, 1313, 1493, 1501, 198
9, 1441, 1616, 1627, 1490, 1732, 1789, 1739, 1789, 1698, 1830, 1873, 1569, 1996, 1533, 1355, 1302, 1771, 1949, 1117, 1137,
 1470, 1101, 1158, 1971, 1104, 1865, 1299, 1560, 1139, 1003, 1113, 1772, 1260, 1826, 1719, 1080, 1378, 1809, 1437, 1060, 1
845, 1080, 1222, 1770, 1894, 1846, 1777, 1361, 1491, 1445, 1325, 1072, 1748, 1288, 1358, 1638, 1638, 1022, 1982, 1976, 165
9, 1323, 1073, 1467, 1043, 1584, 1027, 1272, 1778, 1777, 1261, 1978, 1736, 1130, 1981, 1581, 1183, 1818, 1707, 1325, 1731,
 1093, 1447, 1093, 1190, 1167, 1477, 1929, 1150, 1335, 1347, 1024, 1960, 1373, 1191, 1901, 1317, 1999, 1277, 1894, 1039, 1
638, 1130, 1372, 1743, 1229, 1762, 1591, 1153, 1333, 1572, 1224, 1756, 1436, 1889, 1333, 1471, 1018, 1395, 1554, 1323, 186
0, 1354, 1139, 1013, 1707, 1241, 1654, 1890, 1059, 1044, 1801, 1352, 1183, 1024, 1268, 1228, 1586, 1819, 1735, 1146, 1859,
 1584, 1568, 1306, 1257, 1272, 1226, 1863, 1097, 1978, 1693, 1044, 1665, 1947, 1583, 1819, 1260, 1862, 1134, 1330, 1172, 1
095, 1836, 1830, 1574, 1028, 1756, 1424, 1935, 1343, 1924, 1482, 1042, 1285, 1336, 1453, 1358, 1846, 1397, 1768, 1956, 186
0, 1332, 1493, 1136, 1494, 1364, 1934, 1136, 1603, 1442, 1365, 1549, 1552, 1802, 1096, 1085, 1668, 1732, 1778, 1577, 1588,
 1653, 1256, 1714, 1415, 1454, 1809, 1185, 1493, 1931, 1927, 1437, 1377, 1975, 1739, 1696, 1859, 1545, 1516, 1441, 1395, 1
642, 1976, 1026, 1330, 1168, 1262, 1572, 1316, 1427, 1596, 1957, 1558, 1939, 1729, 1560, 1546, 1563, 1448, 1186, 1078, 119
6, 1981, 1915, 1839, 1120, 1674, 1332, 1449, 1682, 1576, 1748, 1021, 1897, 1171, 1609, 1471, 1345, 1597, 1724, 1071, 1138,
 1340, 1118, 1866, 1850, 1813, 1959, 1376, 1105, 1296, 1887, 1016, 1882, 1472, 1253, 1460, 1706, 1733, 1259, 1517, 1079, 1
431, 1747, 1910, 1141, 1608, 1066, 1400, 1831, 1206, 1743, 1563, 1710, 1851, 1059, 1719, 1532, 1902, 1878, 1751, 1540, 166
8, 1491, 1226, 1782, 1116, 1264, 1910, 1143, 1140, 1115, 1119, 1749, 1854, 1145, 1782, 1086, 1704, 1697, 1888, 1108, 1779,
 1663, 1674, 1502, 1088, 1426, 1828, 1097, 1188, 1844, 1955, 1669, 1357, 1697, 1114, 1182, 1888, 1225, 1557, 1249, 1016, 1
352, 1328, 1768, 1103, 1458, 1371, 1928, 1230, 1179, 1616, 1578, 1564, 1863, 1523, 1058, 1664, 1405, 1440, 1484, 1319, 195
5, 1812, 1659, 1259, 1246, 1599, 1172, 1699, 1841, 1561, 1474, 1985, 1275, 1676, 1831, 1829, 1808, 1607, 1192, 1342, 1100,
 1771, 1978, 1930, 1836, 1096, 1657, 1189, 1953, 1895, 1548, 1414, 1669, 1809, 1219, 1176, 1795, 1092, 1046, 1857, 1806, 1
319, 1816, 1494, 1063, 1809, 1016, 1023, 1696, 1673, 1359, 1594, 1756, 1491, 1893, 1305, 1017, 1781, 1269, 1919, 1121, 109
8, 1961, 1767, 1812, 1583, 1143, 1023, 1625, 1254, 1463, 1725, 1288, 1145, 1459, 1482, 1517, 1745, 1853, 1829, 1355, 1485,
 1127, 1556, 1248, 1093, 1804, 1966, 1653, 1127, 1438, 1774, 1625, 1063, 1528, 1465, 1177, 1146, 1635, 1060, 1208, 1068, 1
034, 1880, 1191, 1594, 1413, 1375]
```

```
Priority: [1, 5, 2, 4, 4, 1, 4, 2, 2, 5, 2, 1, 1, 1, 2, 4, 5, 3, 3, 4, 5, 1, 1, 2, 2, 5, 2, 4, 5, 5, 1, 5, 3, 3, 5, 1, 3,
5, 5, 2, 1, 3, 1, 4, 4, 1, 2, 4, 5, 1, 4, 1, 5, 5, 4, 1, 1, 4, 1, 1, 5, 3, 5, 3, 2, 5, 4, 2, 2, 5, 4, 5, 1, 2, 2, 4, 2, 1,
 2, 5, 1, 3, 2, 4, 2, 5, 2, 2, 4, 1, 4, 1, 3, 1, 1, 1, 1, 1, 3, 5, 5, 4, 5, 2, 5, 3, 3, 1, 3, 5, 4, 4, 3, 5, 3, 3, 3, 3, 1
, 4, 4, 4, 3, 4, 2, 1, 4, 3, 5, 2, 5, 5, 4, 3, 1, 4, 5, 2, 2, 2, 1, 5, 3, 3, 3, 4, 1, 5, 4, 3, 5, 4, 2, 4, 2, 2, 1, 2, 5,
2, 5, 4, 2, 4, 2, 4, 3, 3, 4, 4, 2, 5, 2, 1, 1, 1, 4, 2, 1, 2, 3, 5, 2, 2, 4, 4, 5, 4, 1, 1, 1, 4, 2, 5, 5, 2, 2, 1, 4, 1,
 1, 2, 4, 2, 2, 3, 2, 1, 2, 1, 5, 3, 5, 3, 3, 4, 3, 2, 1, 1, 2, 1, 2, 1, 4, 4, 4, 2, 5, 1, 2, 2, 3, 5, 3, 5, 1, 1, 1, 4, 4
, 2, 2, 5, 1, 5, 1, 4, 1, 1, 2, 5, 5, 3, 4, 5, 3, 5, 2, 1, 4, 4, 3, 1, 4, 1, 3, 5, 5, 3, 3, 4, 1, 2, 1, 4, 2, 1, 4, 4, 5,
4, 5, 5, 2, 4, 2, 3, 1, 2, 1, 1, 3, 2, 5, 4, 5, 4, 5, 2, 3, 4, 1, 1, 3, 2, 1, 5, 2, 5, 3, 2, 2, 3, 4, 3, 3, 2, 3, 3, 3, 2,
 3, 3, 2, 4, 4, 3, 5, 1, 3, 4, 3, 5, 5, 1, 4, 1, 2, 5, 5, 2, 3, 2, 1, 5, 4, 3, 1, 5, 1, 3, 4, 4, 5, 1, 3, 1, 3, 4, 2, 4, 4
, 1, 4, 3, 5, 1, 3, 4, 3, 4, 3, 2, 2, 1, 3, 2, 3, 4, 5, 2, 5, 1, 3, 3, 3, 3, 3, 4, 1, 2, 3, 2, 4, 1, 5, 5, 4, 3, 2, 5, 4,
1, 2, 5, 5, 3, 3, 3, 4, 3, 5, 4, 3, 2, 4, 3, 4, 1, 2, 1, 4, 4, 5, 3, 1, 1, 2, 4, 5, 3, 3, 5, 3, 5, 1, 3, 5, 2, 5, 4, 4, 3,
 5, 5, 5, 3, 1, 2, 4, 5, 3, 1, 4, 3, 3, 2, 3, 5, 3, 2, 5, 1, 1, 5, 5, 5, 5, 1, 3, 4, 2, 3, 5, 3, 3, 3, 3, 3, 1, 3, 5, 5, 1
, 1, 1, 1, 5, 5, 1, 4, 4, 5, 4, 1, 2, 1, 3, 3, 2, 4, 1, 4, 5, 5, 5, 3, 3, 3, 2, 4, 5, 2, 5, 2, 3, 4, 4, 4, 5, 1, 2, 4, 3,
3, 4, 5, 1, 4, 4, 5, 4, 1, 1, 5, 3, 1, 4, 4, 1, 1, 5, 1, 3, 2, 4, 3, 2, 3, 5, 2, 2, 3, 5, 3, 5, 1, 4, 5, 2, 2, 1, 4, 3, 2,
 3, 5, 1, 4, 5, 2, 3, 1, 5, 2, 5, 2, 5, 5, 4, 4, 1, 2, 3, 5, 1, 4, 5, 4, 4, 2, 3, 3, 1, 3, 3, 4, 1, 1, 3, 3, 1, 2, 3, 1, 4
, 3, 4, 4, 3, 1, 3, 5, 1, 4, 1, 1, 1, 3, 4, 1, 2, 2, 2, 3, 2, 1, 2, 1, 2, 4, 4, 3, 1, 2, 1, 1, 4, 1, 4, 4, 5, 5, 1, 5, 3,
2, 4, 2, 2, 4, 4, 1, 1, 5, 5, 1, 4, 4, 2, 2, 2, 2, 3, 2, 3, 1, 5, 3, 5, 2, 1, 1, 3, 4, 4, 1, 3, 2, 2, 3, 2, 2, 1, 3, 2, 5,
 4, 2, 3, 3, 2, 1, 4, 5, 3, 5, 1, 3, 1, 3, 4, 4, 1, 1, 4, 5, 1, 4, 4, 2, 3, 2, 2, 1, 2, 2, 2, 3, 1, 2, 2, 4, 3, 1, 1, 5, 2
, 5, 4, 4, 5, 4, 3, 2, 3, 4, 5, 2, 4, 2, 3, 4, 4, 5, 2, 2, 1, 1, 5, 2, 5, 3, 2, 1, 1, 1, 4, 1, 1, 3, 1, 5, 2, 4, 3, 3, 5,
1, 5, 5, 1, 1, 1, 4, 3, 1, 4, 2, 4, 5, 2, 2, 5, 1, 5, 2, 3, 5, 2, 5, 5, 5, 3, 5, 1, 3, 1, 2, 3, 5, 5, 3, 4, 3, 3, 1, 2, 2,
 3, 5, 1, 2, 3, 3, 5, 1, 3, 1, 5, 3, 4, 1, 5, 2, 1, 1, 3, 1, 2, 4, 2, 4, 3, 5, 4, 3, 3, 5, 3, 4, 4, 5, 2, 3, 5, 4, 3, 2, 5
, 2, 2, 1, 2, 5, 3, 1, 5, 3, 4, 4, 1, 1, 2, 3, 2, 4, 3, 3, 3, 5, 4, 4, 3, 1, 2, 2, 5, 1, 4, 2, 5, 3, 2, 2, 1, 4, 1, 5, 2,
3, 3, 2, 1, 1, 3, 4, 5, 4, 2, 2, 3, 4, 2, 1, 1, 5, 4, 4, 4, 2, 1, 3, 3, 4, 4, 2, 5, 1, 2, 1, 2, 3, 3, 4, 3, 4, 4, 2, 2, 5,
 1, 3, 4, 2, 5, 5, 2, 2, 1, 3, 4, 4, 2, 5, 4, 5, 4, 3, 2, 3, 2, 2, 3, 2, 2, 2, 2, 2, 2, 2, 2, 5, 2, 2, 5, 3, 5, 4, 2, 4, 4
, 2, 5, 1, 4, 4, 5, 4, 4, 3, 3, 5, 1, 1, 3, 4, 1, 1, 4, 2, 2, 3, 2, 3, 2, 4, 1, 4]

Memory:
Pages: [6, 9, 2, 5, 4, 9, 2, 9, 10, 1, 1, 3, 4, 1, 4, 2, 8, 10, 9, 2]
Frames: 5
```

```
Memory:
Pages: [6, 9, 2, 5, 4, 9, 2, 9, 10, 1, 1, 3, 4, 1, 4, 2, 8, 10, 9, 2]
Frames: 5

Disk:
Queue: [105, 102, 64, 167, 113, 172, 28, 128, 192, 165, 112, 98, 56, 140, 44, 53, 177, 111, 108, 84, 18, 194, 135, 110, 17
7, 50, 79, 145, 73, 64, 127, 154, 197, 114, 10, 32, 136, 165, 132, 130, 186, 25, 7, 45, 54, 63, 117, 23, 93, 72]
Head: 119
```

3. Output Generated:

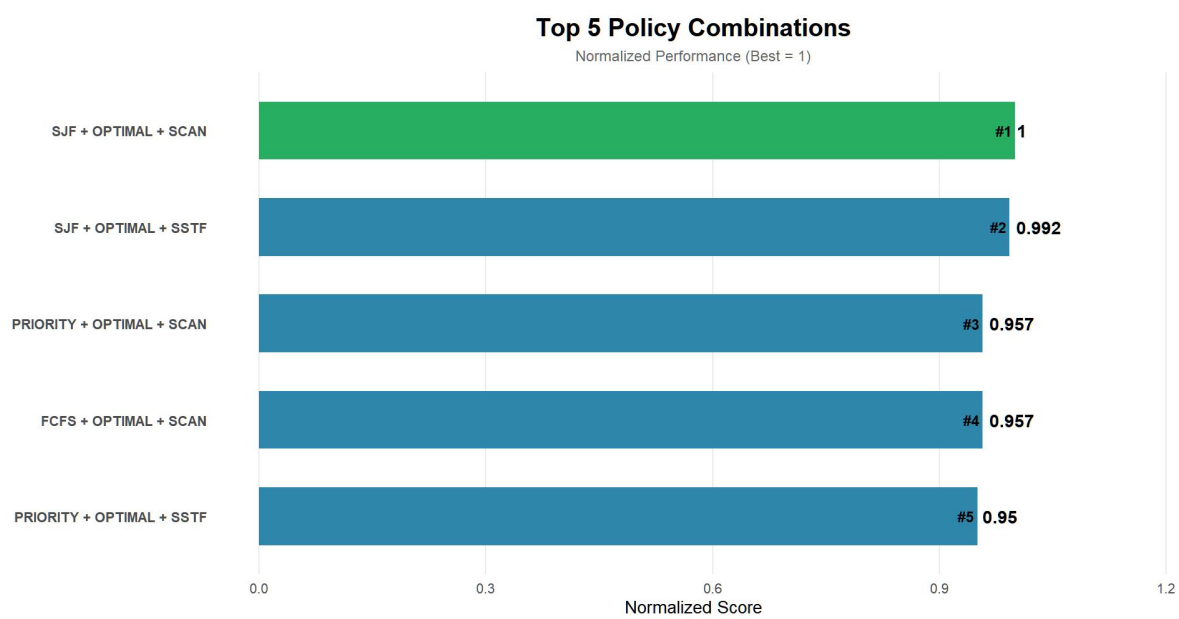
```
Best Policies:
SJF + OPTIMAL + SCAN | WT: 662625.676, PF: 10, ST: 272, Score: 1.0

Insights:

Policy: SJF + OPTIMAL + SCAN
- SJF minimizes waiting time by executing shorter jobs first.
- Optimal minimizes page faults by using future knowledge.
- SCAN provides balanced performance by servicing requests in a direction.
- CPU-bound workload benefits from efficient CPU scheduling.

📁 Results exported to results.csv
 Open plots.R in RStudio to visualize the results
```

4. Reports Generated through R:



CHAPTER 5

CONCLUSION

This project presented a benchmark framework for operating systems that evaluates different combinations of CPU scheduling, page replacement, and disk scheduling algorithms. By simulating various workload types and executing all possible policy combinations, the framework provides a systematic approach to analyze overall system performance.

The results demonstrate that no single algorithm or combination performs optimally for all workloads. Instead, the effectiveness of a policy is highly dependent on the nature of the workload, such as CPU-bound, I/O-bound, or memory-intensive scenarios. By normalizing performance metrics and ranking the results, the framework enables clear comparison and identification of the most suitable policy combinations.

An important outcome of this project is the emphasis on integrated evaluation, rather than analyzing individual algorithms in isolation. This approach provides a more realistic understanding of how different operating system components interact and influence performance. Additionally, the inclusion of insight generation helps explain why certain combinations perform better, making the results more interpretative.

Overall, this framework serves as a practical tool for studying operating system behavior and highlights the importance of workload-aware decision making in selecting efficient resource management strategies. It can be extended further to support advanced optimization techniques and real-world system data for deeper analysis.

REFERENCES

Full Code: https://github.com/supreeth-maraveena/benchmark-framework-for-operating-system/

https://www.geeksforgeeks.org/operating-systems/

https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/

https://www.geeksforgeeks.org/page-replacement-algorithms-in-operating-systems/

https://www.geeksforgeeks.org/disk-scheduling-algorithms/

https://www.geeksforgeeks.org/r-language/data-visualization-with-r-and-ggplot2/

https://www.geeksforgeeks.org/dsa/first-come-first-serve-cpu-scheduling-non-preemptive/

https://www.geeksforgeeks.org/operating-systems/shortest-job-first-or-sjf-cpu-scheduling/

https://www.geeksforgeeks.org/operating-systems/priority-scheduling-in-operating-system/

https://www.geeksforgeeks.org/operating-systems/round-robin-scheduling-in-operating-system/

https://www.geeksforgeeks.org/dsa/scan-elevator-disk-scheduling-algorithms/